

Manual del Programador para la App Athenea

Versión: 1.0.0

Stack: Node.js, Express, PostgreSQL, MongoDB, React Native (Expo)

1. Arquitectura del Proyecto

El sistema sigue un modelo de microservicios. La comunicación se centraliza mediante un API Gateway (Nginx).

Estructura de Directorios

```
/backend
├── /auth-service    # Manejo de JWT, Redis, Postgres
├── /clinical-service # Gestión de Historias Clínicas (Mongo)
├── /ia-service      # Pipeline de Vosk + Groq
/app
/athenea-app
├─ # App React Native (Expo)
│   ├── /src/sync    # Motor de sincronización (SQLite Engine)
│   ├── /src/services # API Consumers
│   └── /src/components# UI Components (Alertas, Wizards)
├── docker-compose.yml # Orquestación de contenedores
└── .env.example       # Template de variables de entorno
```

2. Definición de Servicios y Puertos

Servicio	Función	Puerto	Tecnologías
Auth	Gestión de usuarios y sesiones	3001	Node, Postgres, Redis
Clinical	CRUD Historias Clínicas	3002	Node, Mongoose, Mongo
IA	Pipeline de procesamiento	3003	Python (Vosk), Node
Gateway	Enrutamiento y Balanceo	8080	Nginx

3. Lógica de Persistencia y Datos

Modelo Híbrido

- **Relacional (Postgres):** `users` table.
 - `id`: UUID (PK).
 - `cedula`: VARCHAR(20) UNIQUE.
 - `password_hash`: bcrypt(12 rounds).
- **Documental (MongoDB):**
 - `Historias_Clinicas`: Colección principal.
 - `cedula_paciente`: String (Indexado para búsqueda rápida).
 - Relación: `Historias_Clinicas.cedula_paciente` apunta a la entidad `Pacientes`.

Configuración de Seguridad

- **Autenticación:** stateless mediante JWT. El `id_optometrista` se extrae del payload (`req.usuario.sub`). **Prohibido** tomar ID del `body` del request.
- **Sesión:** Al ejecutar `logout`, el cliente debe invocar `borrarTodasLasHistorias()` en SQLite para evitar fugas de información entre sesiones en un mismo dispositivo.

4. Motor de Sincronización (Offline-First)

El módulo `syncEngine.js` actúa como middleware entre el almacenamiento local y la nube.

Algoritmo de Sincronización:

1. **Read:** `db.executeSql('SELECT * FROM historias_pendientes WHERE sincronizado = 0')`.
2. **Post:** Loop sobre registros pendientes. Request `POST /clinical-service/upload`.
3. **Verify:** Esperar respuesta `201 Created`.
4. **Commit:** `db.executeSql('UPDATE historias_pendientes SET sincronizado = 1 WHERE id = ?')`.
5. **Error Handling:** Si la red cae, el flag `sincronizado` permanece en `0` para reintento automático.

5. Pipeline de Procesamiento IA

La conversión de audio a datos clínicos sigue este flujo:

1. **Entrada:** Formato `M4A` (Cliente).
2. **Transformación:** `ffmpeg -i input.m4a -ar 16000 -ac 1 output.wav`.
3. **Transcripción:** `child_process.spawn('python3', ['transcribir.py', 'output.wav'])`.
 - *Dependencia:* `vosk-model-small-es-0.42`.
4. **Estructuración:** Envío a Groq `llama-3.3-70b-versatile`.
 - *Constraint:* `temperature: 0.1` (Minimización de alucinaciones).
 - *Fallback:* Regex local en `parser.js`.

6. Resolución de Problemas

Bug: Valores de refracción no cargan

- **Causa:** Validación imprecisa (`if (ref.esf_od)`). El valor `-0.25` se evalúa como `truthy` pero la lógica de validación de formulario fallaba con negativos.
- **Solución:** Usar validación explícita:
- JavaScript

```
if (ref.esf_od != null && ref.esf_od !== "") { ... }
```

-
-

Bug: Logout no redirige

- **Causa:** Stack Navigator con `initialRouteName` estático.
- **Solución:** Renderizado condicional en el archivo `App.js` o `Navigation.js`:
- JavaScript

```
{!token ? <AuthStack /> : <MainStack />}
```

-
-

Bug: Nombre del especialista no actualiza

- **Causa:** `useFocusEffect` no dispara en el primer montaje.
- **Solución:** Implementar `useEffect` con array de dependencias vacío `[]` para la carga inicial de datos de perfil.